



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

03-深度神经网络量化

贾弘洋 & 刘勇攀

2025/03/4





- Artificial intelligence, machine learning, and deep learning
- Building a machine learning algorithm:
 - Dataset
 - Cost function
 - Optimization function
 - Model
- Deep learning to automatically extract hierarchical data
 - Better at handle high-dimensional data
 - Key differences are in the Model
 - Multiple layers, i.e., deep
 - Linear + non-linear layers



- Classification App
 - Dataset: ImageNet
 - AlexNet/VGG/ResNet
- Detection App
 - Dataset: COCO
 - Two-stage: R-CNN, SSD
 - One-stage: YOLO
- Serial App
 - Dataset
 - RNN
 - LSTM
 - Transformers

/ Overview



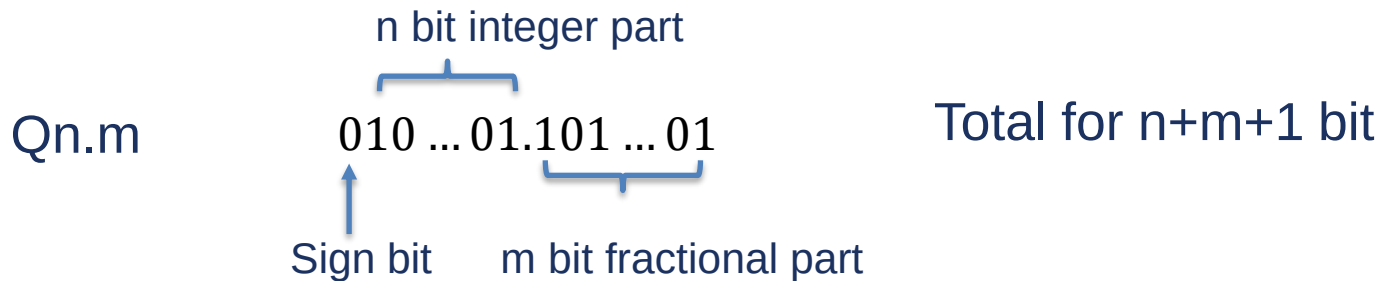
- In today's lecture, we will look at one important and widely-used techniques to design an efficient digital system – quantization.
- We will first introduce numerical representation methods. High-precision representation methods could lead to a costly implementation on hardware. Low-precision representation may cause accuracy loss.
- We will learn how to implement a quantized model on hardware with due precision efficiently.
- State-of-the-art quantization methods for deep learning will be introduced and discussed.



- Fixed-point and floating-point representation
- Hardware implications
- Quantization on deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

/ Fixed-point arithmetic

- Represent a number in fixed-point



- Example: 4.23 in binary?

Q _{4.2}	00100.01 (4.25)
Q _{4.5}	00100.00111 (4.21875)
Q _{4.8}	00100.00111011 (4.23046875)

/ Fixed-point arithmetic



- Fixed-point with slope and bias
 - Scaling and bias: help represent in different range and precision

$$y = s * x + z$$

- Example: 3 bit fixed-point representation with slope and bias

$s = 1, z = 0$

2^2	2^1	2^0	Val
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

$s = 1/4, z = 0$

2^0	2^{-1}	2^{-2}	Val
0	0	0	0
0	0	1	1/4
0	1	0	2/4
0	1	1	3/4
1	0	0	1
1	0	1	5/4
1	1	0	6/4
1	1	1	7/4

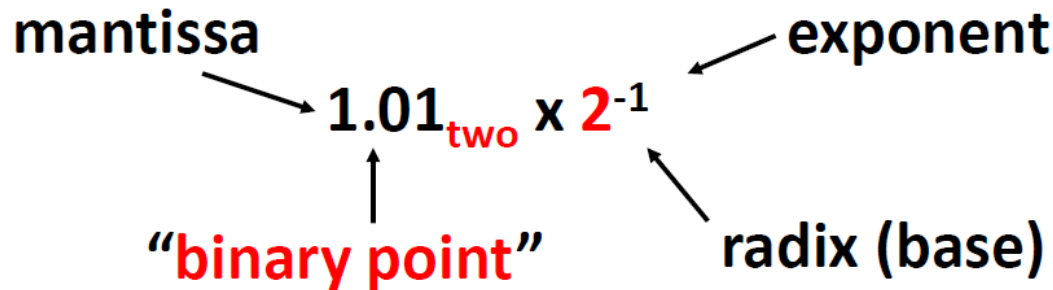
$s = 4, z = 0$

2^4	2^3	2^2	Val
0	0	0	0
0	0	1	4
0	1	0	8
0	1	1	12
1	0	0	16
1	0	1	20
1	1	0	24
1	1	1	28

$s = 1.5, z = 10$

2^2	2^1	2^0	Val
0	0	0	$1.5*0+10$
0	0	1	$1.5*1+10$
0	1	0	$1.5*2+10$
0	1	1	$1.5*3+10$
1	0	0	$1.5*4+10$
1	0	1	$1.5*5+10$
1	1	0	$1.5*6+10$
1	1	1	$1.5*7+10$

/ Floating-point arithmetic

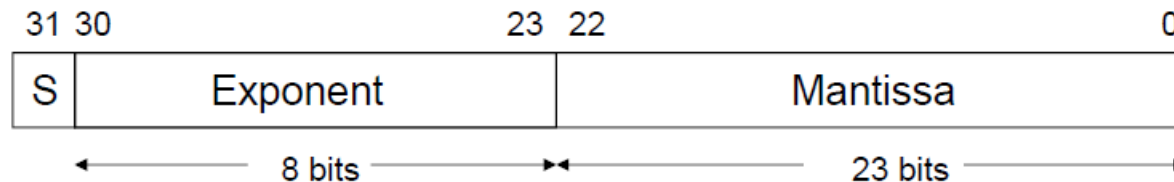


- Computer arithmetic that supports it called floating point, because it represents numbers where the binary point is not fixed, as it is for integers

/ Floating-point arithmetic



- Floating point representation (FP32)



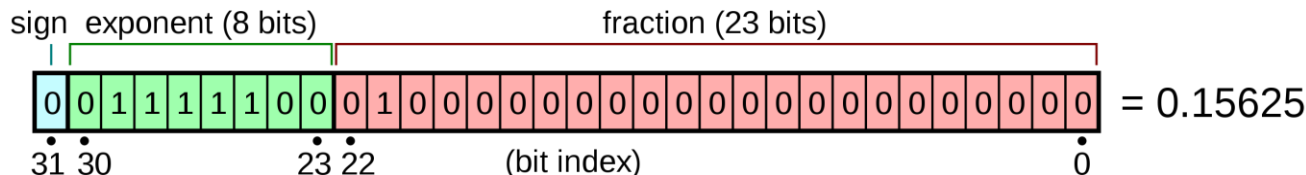
$$(-1)^S * M * 2^E$$

- Represent range:

$$X.Y * 10^{-38} \sim X.Y * 10^{38}$$

/ Floating-point arithmetic

- Floating point representation (FP32)



- Represent range:

Exponent	fraction = 0	fraction $\neq$ 0	Equation
$00_H = 00000000_2$	$\pm zero$	subnormal number	$(-1)^{sign} \times 2^{-126} \times 0.fraction$
$01_H, \dots, FE_H = 00000001_2, \dots, 11111110_2$	normal value		$(-1)^{sign} \times 2^{exponent-127} \times 1.fraction$
$FF_H = 11111111_2$	$\pm infinity$	NaN (quiet, signalling)	

$$1.4 * 10^{-45} (subnormal) / 1.18 * 10^{-38} (normal) \sim 1.7 * 10^{38}$$

/ Floating-point arithmetic

- IEEE 754 Floating Point Standard
 - Called Biased Notation, where bias is number subtracted to get real number.
IEEE 754 **uses bias** of 127 for single precision, 1023 for double precision.

31 _o	30 _o	29 _o	... _o	... _o	24 _o	23 _o	22 _o	21 _o	... _o	... _o	1 _o	0 _o
0 _o	10000110 _o						0110010001 _o					
S _o	Exponent (8 bit) _o						Mantissa (23 bit) _o					

Floating-Point Format: $(-1)^S \times (1 + M) \times 2^{(E-Bias)}$

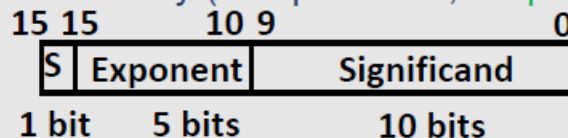
- Example: 178.125 in FP32?

$$178.125 = 10110010.001 = 1.0110010001 \times 2^{111} = 1.0110010001 \times 2^{10000110-011111111}$$

/ Floating-point arithmetic

- IEEE 754 Floating Point Standard
 - Called Biased Notation, where bias is number subtracted to get real number.
IEEE 754 uses bias of 15 for half precision

- Summary (half precision, or fp15):



- $(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-15)}$



- Fixed-point and floating-point representation
- Hardware implications
- Quantization on deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

NVIDIA GPU 的算力提升

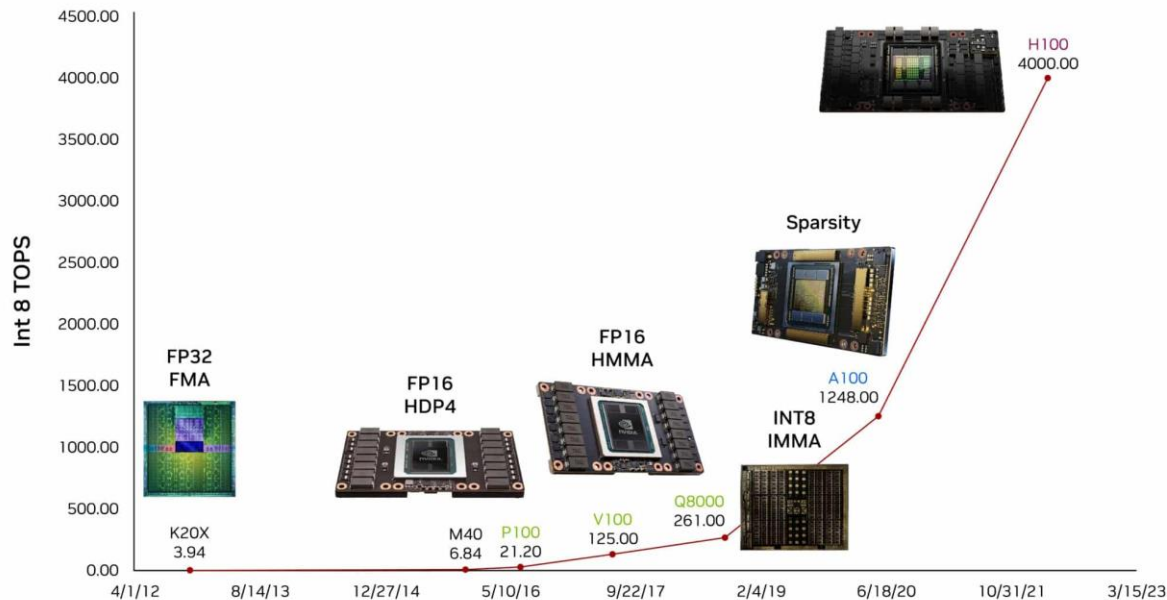


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Gains from

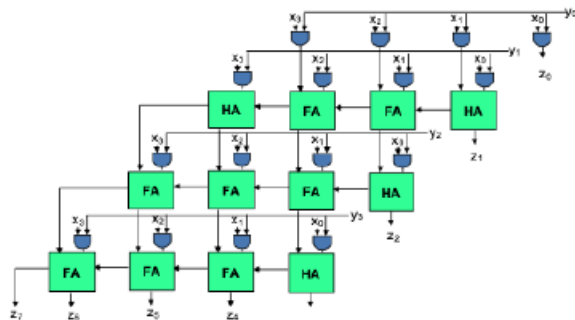
- Number Representation
 - FP32, FP16, INT8
 - (TF32, BF16)
 - ~16x
- Complex Instructions
 - DP4, HMMA, IMMA
 - ~12.5x
- Process
 - 28nm, 16nm, 7nm, 5nm
 - ~2.5x
- Sparsity
 - ~2x
- Model efficiency has also improved – overall gain > 1000x

Single-Chip Inference Performance - 1000X in 10 Years



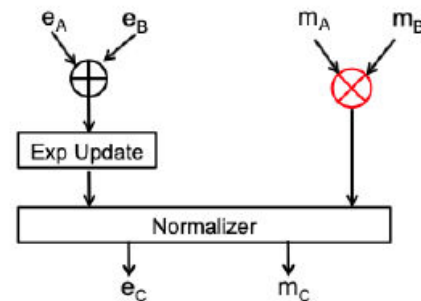
[Bill Dally, Trends in Deep Learning Hardware]

量化的硬件基础



Fixed-Point Multiplier

Multiplier Example: $C = A \times B$



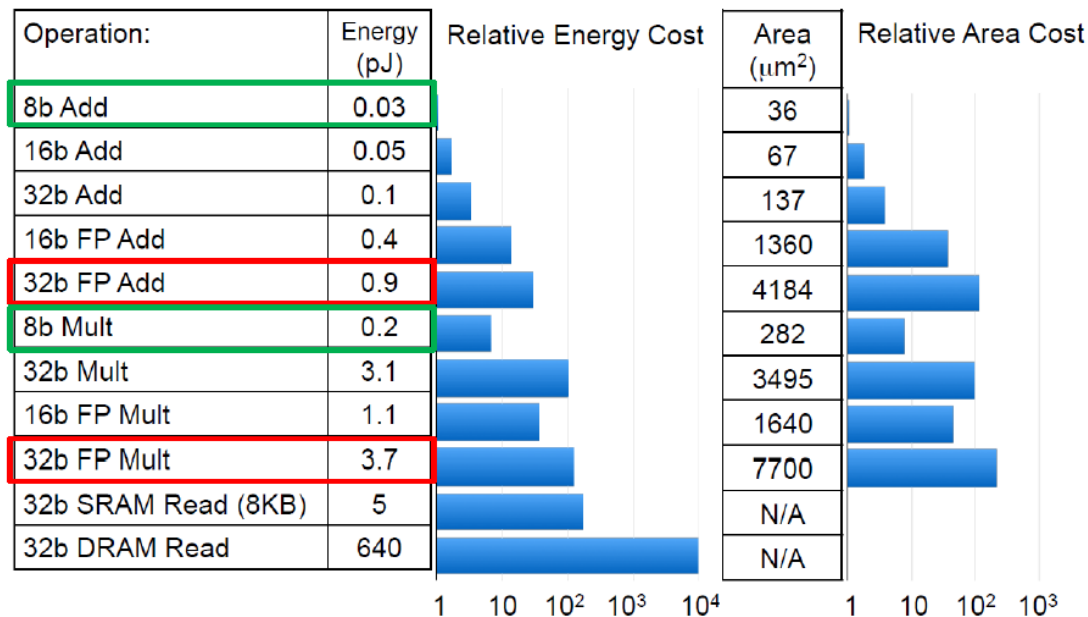
Floating-Point Multiplier

- FA: full-adder
- HA: half-adder

量化的硬件基础

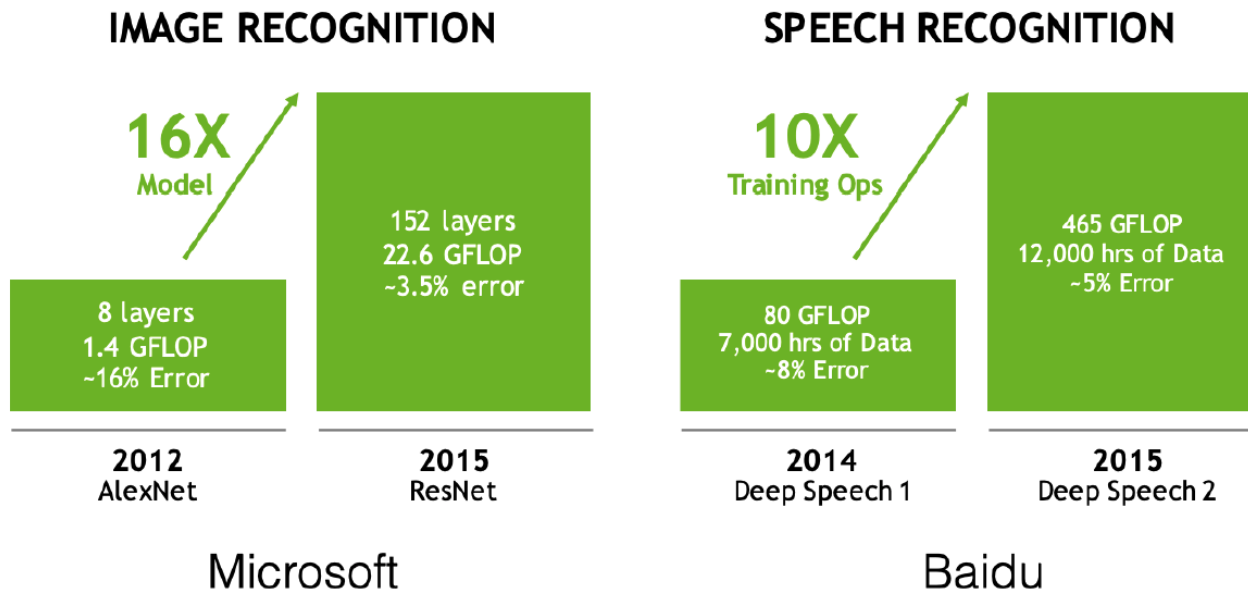


- What we need?
 - Adequate precision, low power consumption, fewer area



Question: do floating-point the only choice for all cases?

- Let's return to the question: why we need quantization in deep learning?



/量化的硬件基础



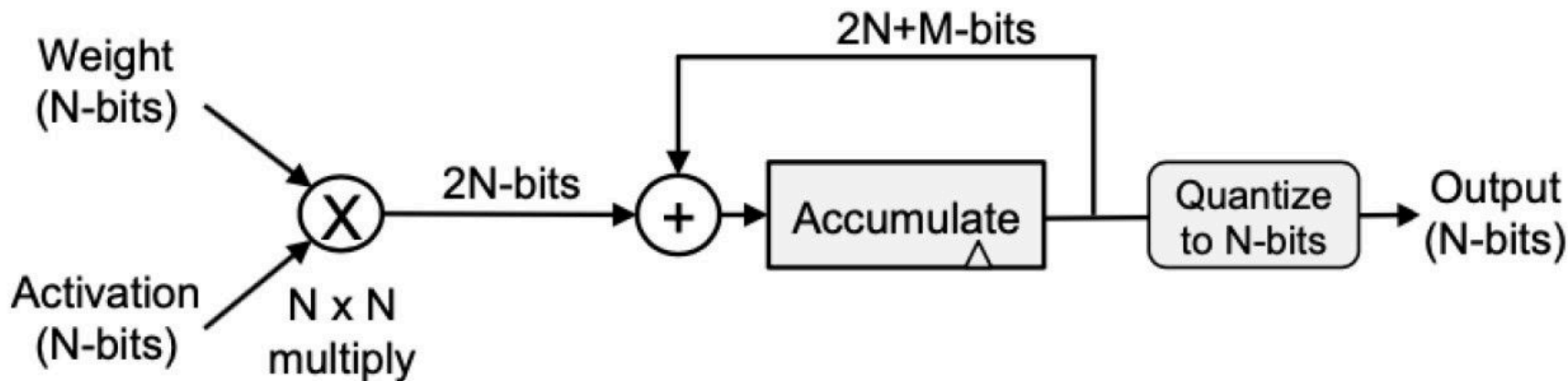
- Cases on image classification (Narayanan@DAC2020)

task	Accuracy on FP32	Accuracy on 8 bit Fixed-point	Degradation
ResNet20 on Cifar10	92.34	92.28	0.06
ResNet18 on ImageNet	70.1	67.5	2.6

If we take some tricks, 8 bit fixed-point is enough!

量化的硬件基础

- Low bit Fixed-point representations on digital system
 - Accumulation requires higher precision than input/output!





- Fixed-point and floating-point representation
- Hardware implications
- Quantization for deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

/ Quantization for deep learning



- Two mainstream approaches
 - **Post-training quantization**: quantize a pre-trained model to maintain its accuracy. Actually, it is very challenging because FP32 has far larger representation range and higher precision than 8-bit fixed-point.
 - **Quantization-aware training**: train to get a quantized model. It is even more challenging.

/ Post-training Quantization



- Quantization scheme:

- The correspondence between the fixed-point representation of values, i.e., **“q” for “quantized value”** and their floating-point value, i.e., **“r” for “real value”**.
- Recall “slope and bias” of fixed-point representation: $y = s \cdot x + z$

$$r = S(q - Z)$$

```
template<typename QType> // e.g. QType=uint8
struct QuantizedBuffer {
    vector<QType> q;        // the quantized values
    float S;               // the scale
    QType Z;               // the zero-point
};
```

r: real floating-point value
q: quantized fixed-point value
S: scaling factor
Z: zero point (bias)

/ Post-training Quantization



- Quantization scheme

$$OA[i, k] = \sum_{j=1}^N (W[i, j] * IA[j, k]) \quad r = S(q - Z)$$

Diagram illustrating the quantization scheme. The equation shows the output $OA[i, k]$ (labeled "output" with an upward arrow) is calculated as the sum over $j=1$ to N of the product of the weight $W[i, j]$ (labeled "Weight in DNN" with an upward arrow) and the input $IA[j, k]$ (labeled "input" with a downward arrow).

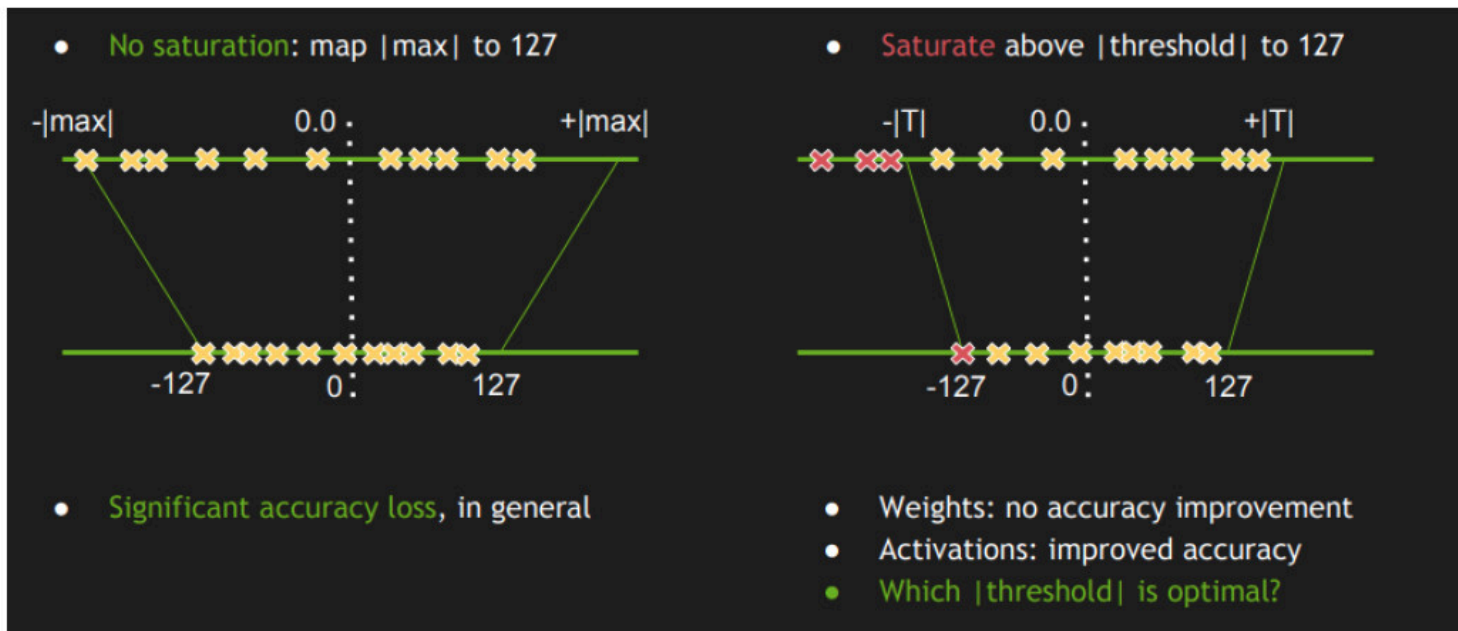
$$S_{OA} (q_{OA}^{(i,k)} - Z_{OA}) = \sum_{j=1}^N (S_W (q_W^{(i,j)} - Z_W) * (S_{IA} (q_{IA}^{(j,k)} - Z_{IA})))$$

$$q_{OA}^{(i,k)} = Z_{OA} + \frac{S_W * S_{IA}}{S_{OA}} \sum_{j=1}^N ((q_W^{(i,j)} - Z_W) * (q_{IA}^{(j,k)} - Z_{IA}))$$

$$q_{OA}^{(i,k)} = \frac{S_W * S_{IA}}{S_{OA}} \sum_{j=1}^N (q_W^{(i,j)} * q_{IA}^{(j,k)}) \quad (\text{If we neglect the zero point (bias)})$$

/ Post-training Quantization

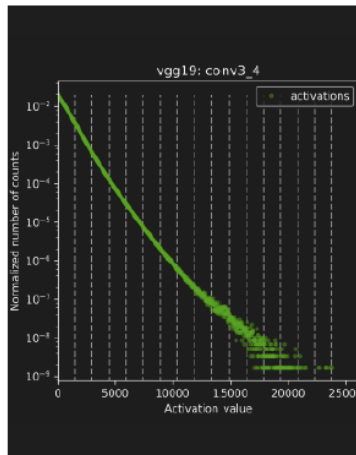
- The essential problem: how can we determine the scaling factor of input, output and weight in quantizing DNNs?



/ Which threshold is optimal?



- Again, it depends.
 - A lot of research/engineering practices in this area, e.g.,
 - Averaging: using a (weighted) average of min/max values of samples in the batch instead of the global min/max
 - Mean $\pm$ N * std: Take N standard deviation from the mean
 - Calibration (TensorRT)

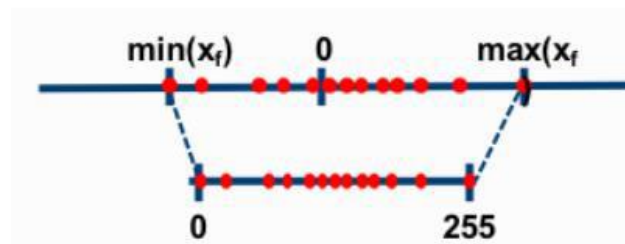


- Run FP32 inference on Calibration Dataset.
- For each Layer:
 - collect histograms of activations.
 - generate many quantized distributions with different saturation thresholds.
 - pick threshold which minimizes $KL_divergence(ref_distr, quant_distr)$.
- Entire process takes a few minutes on a typical desktop workstation.

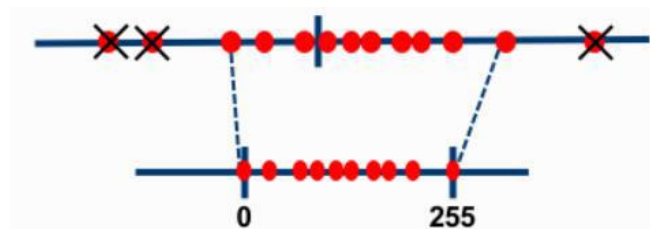
Post-training Quantization



- Asymmetric mapping



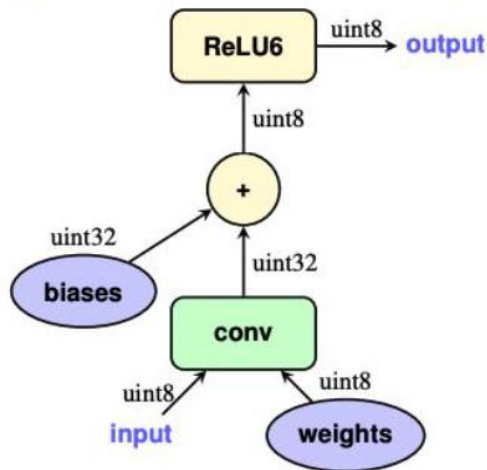
- Removing outliers



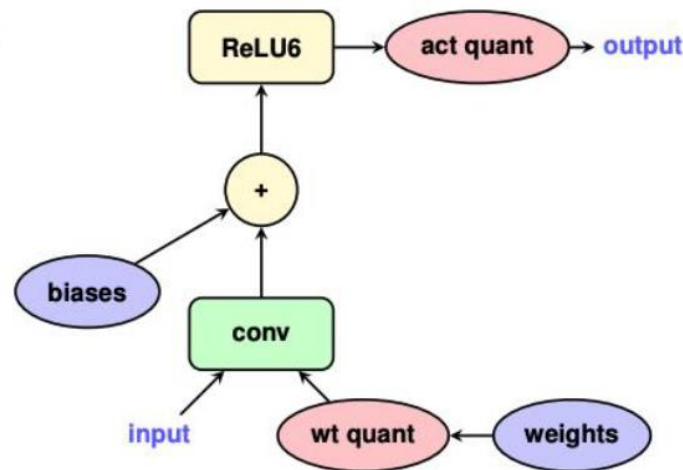
/ Quantization-aware training



- Typically performs better than post-training quantization
- “Simulate” quantization effects in the forward pass
- Weights and biases are updated in floating point during backpropagation so that they can be nudged by small amounts.



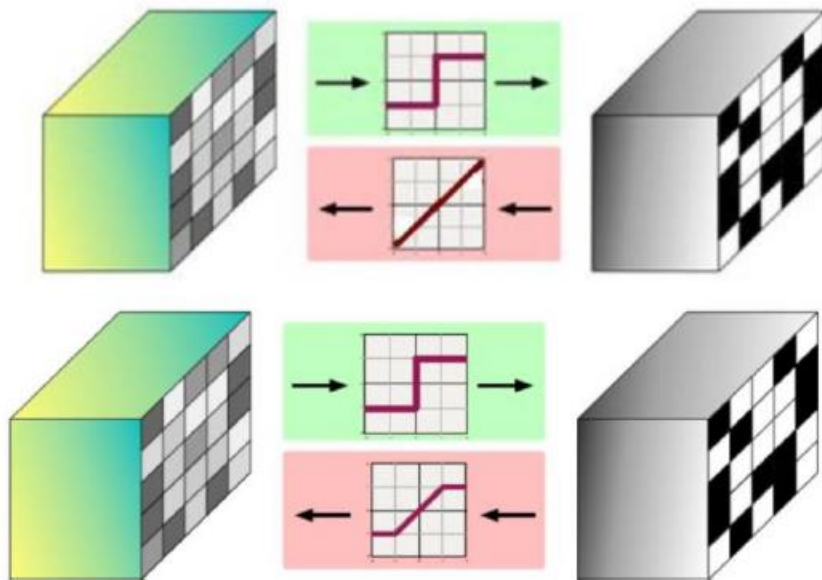
(a) Integer-arithmetic-only inference



(b) Training with simulated quantization

/ Quantization-aware training

- As the quantization procedure is not differentiable, a STE (直通估计器) module is needed



Direct STE: pass gradient directly

Saturated STE: clip the gradient



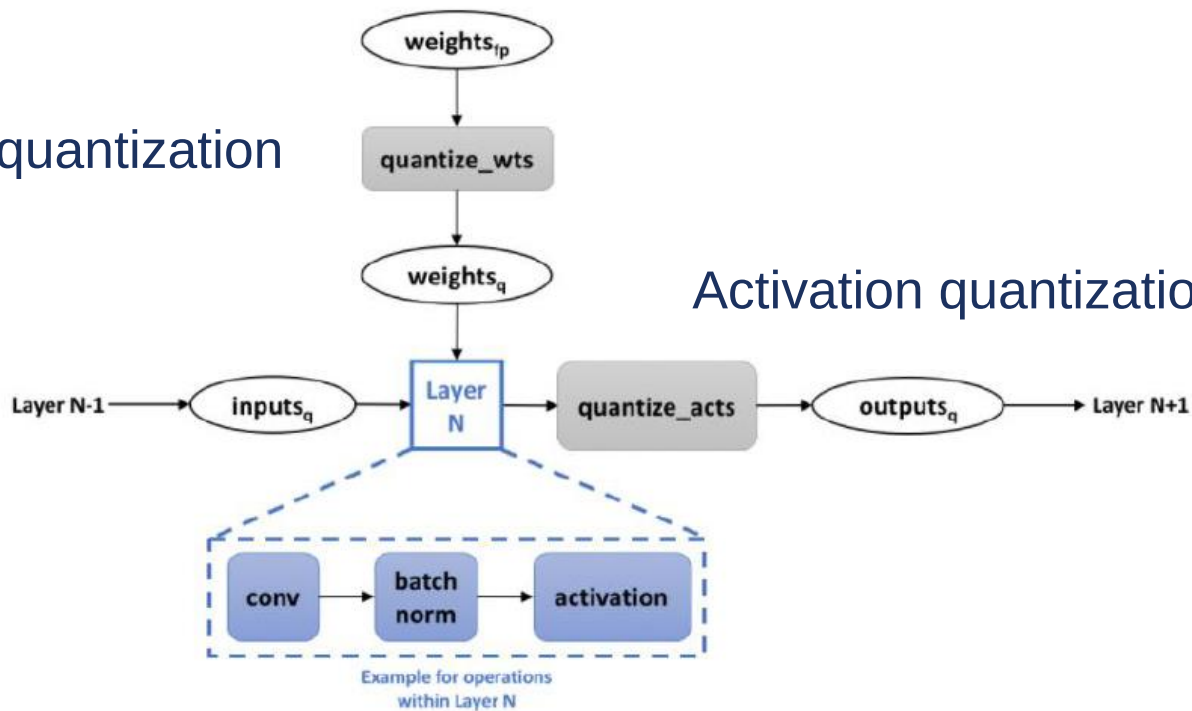
- Fixed-point and floating-point representation
- Hardware implications
- Quantization for deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

/ Basic Structure



Weight quantization

Activation quantization



/DoReFa-Net



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

- Main Contribution
 - First introduce the quantization for gradient in the training phase.
(think about what is the benefit?)

Paper site: <https://arxiv.org/pdf/1606.06160.pdf>

- Technical Details
 - Normalize the data before quantize weights. The purpose is to ensure the quantization will not change the data distribution.

$$\textbf{Forward: } r_o = f_{\omega}^k(r_i) = 2 \text{quantize}_k\left(\frac{\tanh(r_i)}{2 \max(|\tanh(r_i)|)} + \frac{1}{2}\right) - 1.$$

$$\textbf{Backward: } \frac{\partial c}{\partial r_i} = \frac{\partial r_o}{\partial r_i} \frac{\partial c}{\partial r_o} \quad 4$$

- Technical Details
 - Introduce uniform noise to offset the quantization noise for gradient.

$$N(k) = \frac{\sigma}{2^k - 1} \text{ where } \sigma \sim \text{Uniform}(-0.5, 0.5)$$

$$f_{\gamma}^k(dr) = 2 \max_0(|dr|) \left[\text{quantize}_k \left[\frac{dr}{2 \max_0(|dr|)} + \frac{1}{2} + N(k) \right] - \frac{1}{2} \right]$$

$$\textbf{Backward: } \frac{\partial c}{\partial r_i} = f_{\gamma}^k \left(\frac{\partial c}{\partial r_o} \right)$$

- Technical Details

- Replace multiply accumulate with bitcount operations.

Suppose $\mathbf{x} = \sum_{m=0}^{M-1} c_m(\mathbf{x})2^m$ and $\mathbf{y} = \sum_{k=0}^{K-1} c_k(\mathbf{y})2^k$, where $c_m(x)$ and $c_m(y)$ are bit vectors. Then, the multiply of $\mathbf{x}$ and $\mathbf{y}$ can be defined as

$$\mathbf{x} \cdot \mathbf{y} = \text{bitcount}(\text{and}(\mathbf{x}, \mathbf{y})), x_i, y_i \in \{0, 1\} \forall i.$$

or

$$\mathbf{x} \cdot \mathbf{y} = \sum_{m=0}^{M-1} \sum_{k=0}^{K-1} 2^{m+k} \text{bitcount}[\text{and}(c_m(\mathbf{x}), c_k(\mathbf{y}))],$$
$$c_m(\mathbf{x})_i, c_k(\mathbf{y})_i \in \{0, 1\} \forall i, m, k.$$

- Results and extra findings

W	A	G	Training Complexity	Inference Complexity	Storage Relative Size	AlexNet Accuracy
1	1	6	7	1	1	0.395
1	1	8	9	1	1	0.395
1	1	32	-	1	1	0.279 (BNN)
1	1	32	-	1	1	0.442 (XNOR-Net)
1	1	32	-	1	1	0.401
1	1	32	-	1	1	0.436 (initialized)
1	2	6	8	2	1	0.461
1	2	8	10	2	1	0.463
1	2	32	-	2	1	0.477
1	2	32	-	2	1	0.498 (initialized)
1	3	6	9	3	1	0.471
1	3	32	-	3	1	0.484
1	4	6	-	4	1	0.482
1	4	32	-	4	1	0.503
1	4	32	-	4	1	0.530 (initialized)
8	8	8	-	-	8	0.530
32	32	32	-	-	32	0.559

W: weight

A: activation

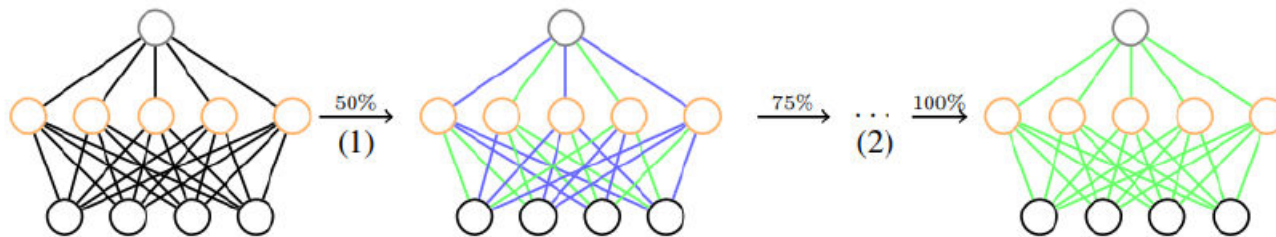
G: gradient

In the training-aware quantization, sensitivity level towards numerical precision: $G > A > W$

We should first ensure the precision of gradient

- Contributions
 - First introduce an iteration training methods for network quantization.
The training methods can effectively maintain the accuracy for low bit quantization.
 - The method quantizes weights to 2^n , thus especially suitable for hardware utilization

- Training methods
 - Divide the weights into 2 groups. Quantize 50% of them and freeze them in the following training. Train the rest 50% weights to recover model accuracy.
 - Repeat the above procedure for the rest 50% weights.



0.01	0.02	-0.20	0.04	0.33
0.17	-0.42	-0.33	0.02	-0.05
0.02	0.83	-0.03	0.03	0.06
-0.90	0.07	0.11	0.87	-0.36
-0.73	0.41	0.42	0.39	0.47



0.01	0.02	-0.20	0.04	2^{-2}
0.17	2^{-1}	2^{-2}	0.02	-0.05
0.02	2^0	-0.03	0.03	0.06
2^0	0.07	0.11	2^0	2^{-2}
2^{-1}	2^{-1}	2^{-1}	2^{-1}	2^{-1}



0.11	0.04	-0.7	0.19	2^{-2}
0.15	2^{-1}	2^{-2}	-0.09	-0.02
-0.02	2^0	-0.06	0.21	0.15
2^0	0.27	-0.09	2^0	2^{-2}
2^{-1}	2^{-1}	2^{-1}	2^{-1}	2^{-1}



2^{-3}	0.03	2^{-1}	2^{-2}	2^{-2}
2^{-3}	2^{-1}	2^{-2}	-0.13	-0.01
-0.03	2^0	-0.11	2^{-2}	2^{-3}
2^0	0.091	-0.01	2^0	2^{-2}
2^{-1}	2^{-1}	2^{-1}	2^{-1}	2^{-1}



2^{-3}	-0.05	2^{-1}	2^{-2}	2^{-2}
2^{-3}	2^{-1}	2^{-2}	2^{-3}	-0.02
0.02	2^0	2^{-3}	2^{-2}	2^{-3}
2^0	2^{-3}	-0.04	2^0	2^{-2}
2^{-1}	2^{-1}	2^{-1}	2^{-1}	2^{-1}



2^{-3}	0	2^{-1}	2^{-2}	2^{-2}
2^{-3}	2^{-1}	2^{-2}	2^{-3}	0
0	2^0	2^{-3}	2^{-2}	2^{-3}
2^0	2^{-3}	0	2^0	2^{-2}
2^{-1}	2^{-1}	2^{-1}	2^{-1}	2^{-1}

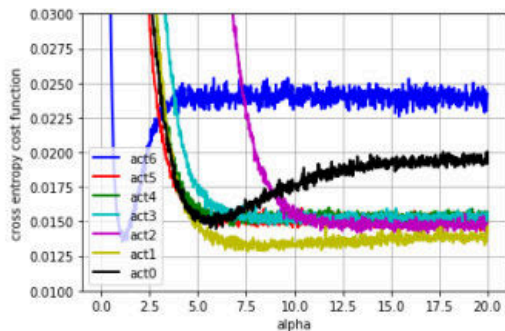
- Repeatedly sample half of the rest weights to quantize
- Quantize the weights into 2^n

- Training details
 - What kind of weight should be quantized first before other weights?
 - How to freeze the quantized weight in the following training phase?

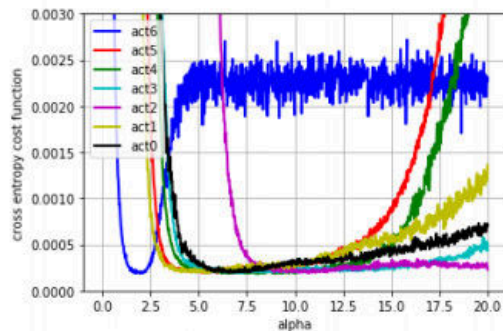
- Main contributions
 - Clipping the activation before quantization can improve the performance. (think about why?)
- Clip the activation:

$$y = PACT(x) = 0.5(|x| - |x - \alpha| + \alpha) = \begin{cases} 0, & x \in (-\infty, 0) \\ x, & x \in [0, \alpha) \\ \alpha, & x \in [\alpha, +\infty) \end{cases}$$

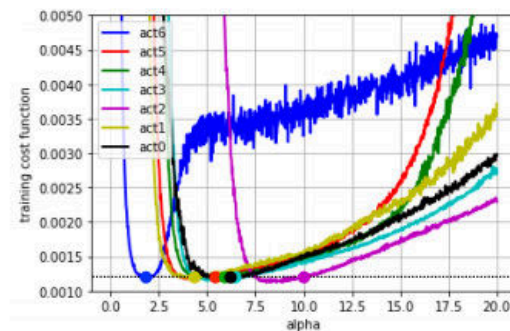
- Different layers need different α



(a) Cross-entropy in full-precision



(b) Cross-entropy with quantization

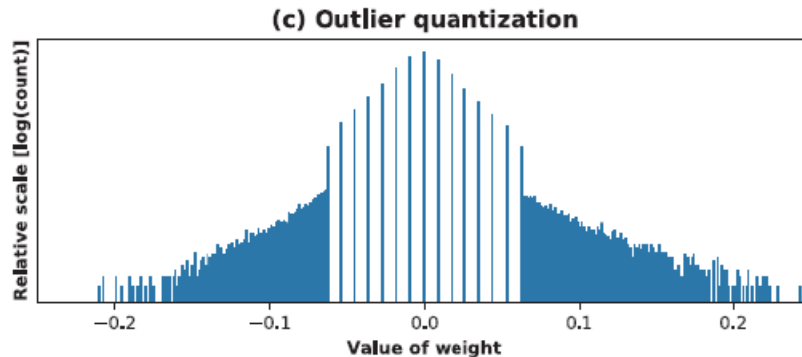
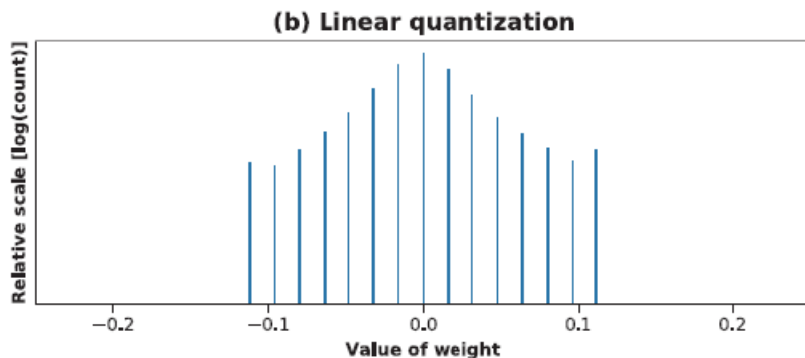


(c) Training loss

- α should be learnable

/ Outlier quantization

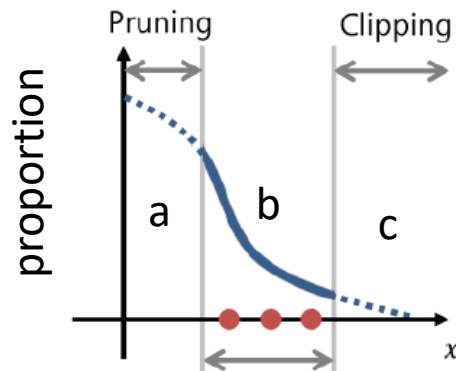
- Main contributions
 - Use different quantized bits to quantize different weights. Some weight can have higher precision while others not.



Paper site: <https://ieeexplore.ieee.org/document/8416865>

/ Quantization interval learning

- Main contributions
 - Most of the weights are very small. Minor weights can have too large value. This two kinds of weights make it hard for fixed-point quantization (think about why?)



- a: small weights, can be pruned directly
- b: weights worth quantizing
- c: large weights, should be clipped into b area during training

/ Quantization interval learning



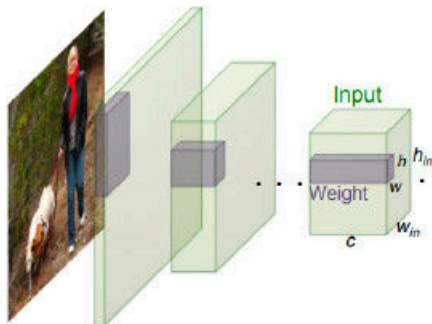
- Experiment results

Method	ResNet-18 (FP: 70.2)				ResNet-34 (FP: 73.7)				AlexNet (FP: 61.8)			
	Bit-width (A/W)											
	5/5	4/4	3/3	2/2	5/5	4/4	3/3	2/2	5/5	4/4	3/3	2/2
QIL (Ours)[†]	70.4	70.1	69.2	65.7	73.7	73.7	73.1	70.6	61.9	62.0	61.3	58.1
LQ-Nets [26]	-	69.3	68.2	64.9	-	-	71.9	69.8	-	-	-	57.4
PACT [4]	69.8	69.2	68.1	64.4	-	-	-	-	55.7	55.7	55.6	55.0
DoReFa-Net [27]	68.4	68.1	67.5	62.6	-	-	-	-	54.9	54.9	55.0	53.6
ABC-Net [17]	65.0	-	61.0	-	68.4	-	66.7	-	-	-	-	-
BalancedQ [28]	-	-	-	59.4	-	-	-	-	-	-	-	55.7
TSQ [†] [25]	-	-	-	-	-	-	-	-	-	-	-	58.0
SYQ [†] [6]	-	-	-	-	-	-	-	-	-	-	-	55.8
Zhuang <i>et al.</i> [30]	-	-	-	-	-	-	-	-	-	58.1	-	52.5
WEQ [20]	-	-	-	-	-	-	-	-	-	55.9	54.9	50.6

/ Binary neural networks (BNN)



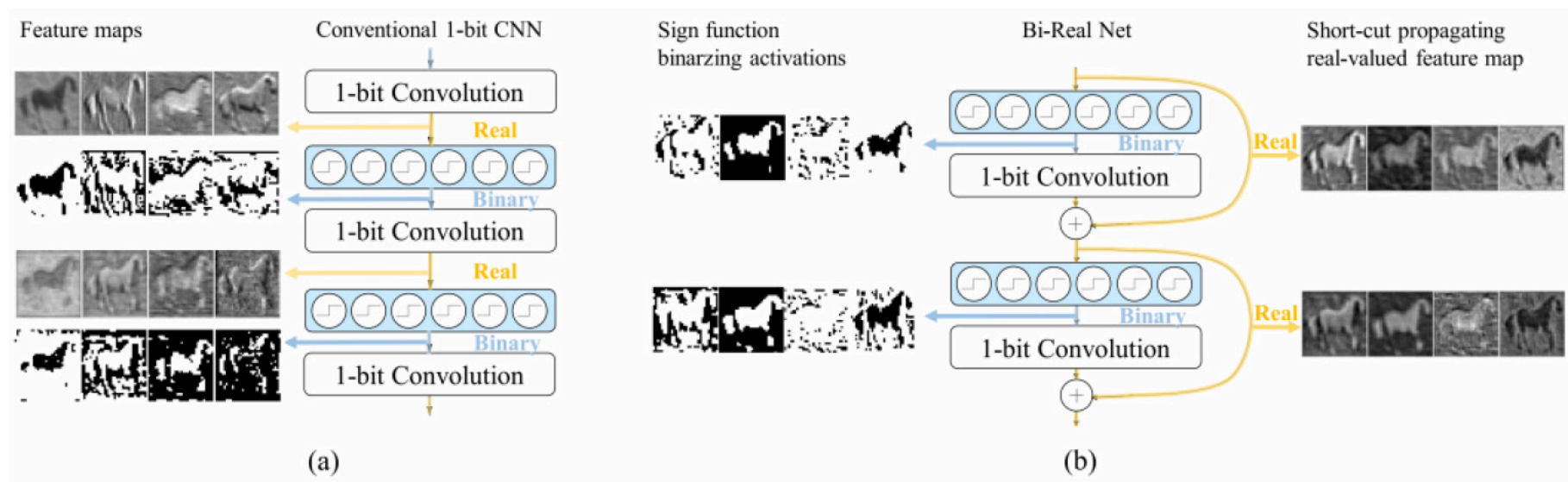
- Networks with weights composed of $\{-1, 1\}$



	Network Variations	Operations used in Convolution	Memory Saving (Inference)	Computation Saving (Inference)	Accuracy on ImageNet (AlexNet)
Standard Convolution	Real-Value Inputs 0.11 -0.21 ... -0.34 -0.25 0.61 ... 0.52 Real-Value Weights 0.12 -1.2 ... 0.41 -0.2 0.5 ... 0.68	$+, -, \times$	1x	1x	%56.7
Binary Weight	Real-Value Inputs 0.11 -0.21 ... -0.34 -0.25 0.61 ... 0.52 Binary Weights 1 -1 1 -1 1 1	$+, -$	$\sim 32x$	$\sim 2x$	%56.8
BinaryWeight Binary Input (XNOR-Net)	Binary Inputs 1 -1 ... -1 -1 1 ... 1 Binary Weights 1 -1 1 -1 1 1	XNOR , bitcount	$\sim 32x$	$\sim 58x$	%44.2

Binary neural networks (BNN)

- Very few existing researches: BinaryNet, XNOR-Net, MeliusNet

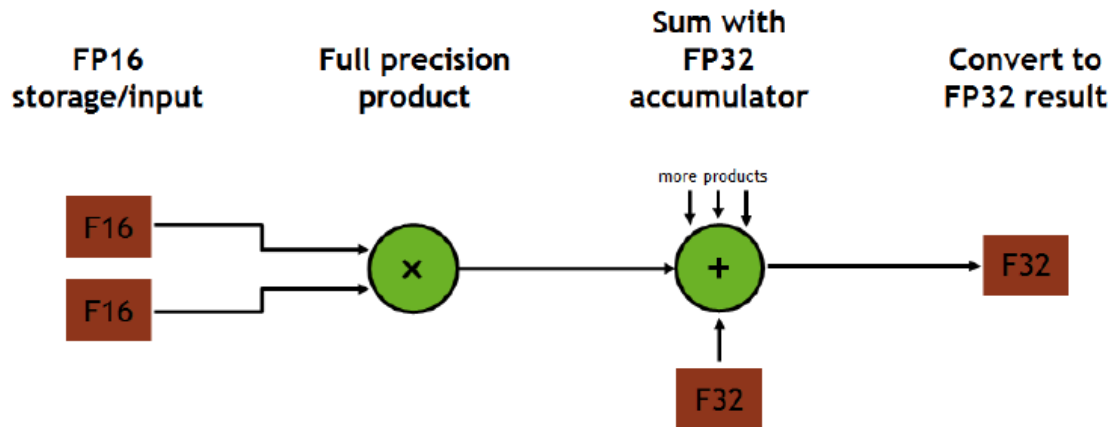




- Fixed-point and floating-point representation
- Hardware implications
- Quantization for deep learning
- Classic research for quantization methods
- State-of-the-art hardware support for low precision DNNs

/ GPU tensor cores

E.g. First Generation
Nvidia Volta GPU



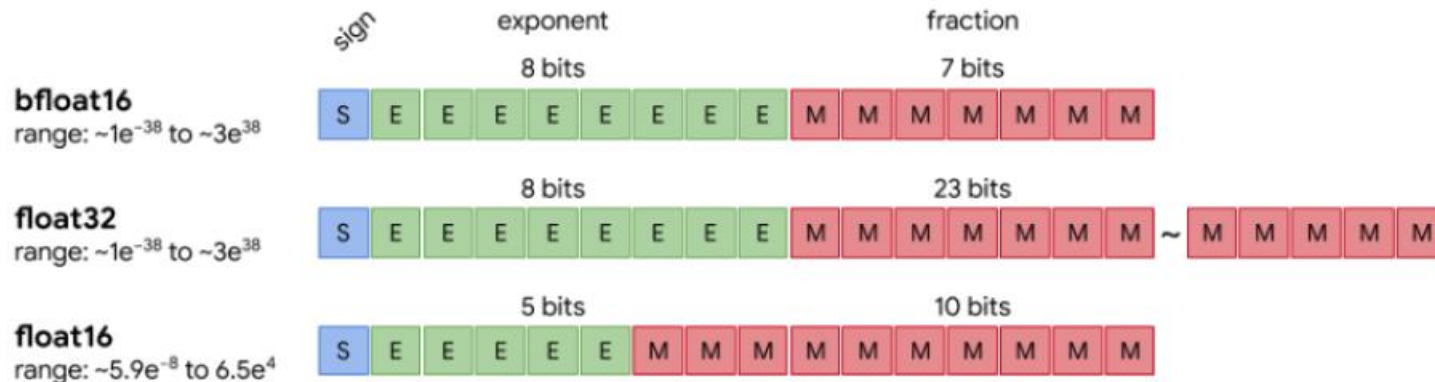
- <https://www.nvidia.com/en-us/data-center/tensor-cores/>
- <https://devblogs.nvidia.com/programming-tensor-cores-cuda-9/>
 - Used by cuDNN and CUBLAS libraries
 - Exposed in CUDA as WMMA
 - <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma>
- Accelerate convolutions and matrix multiplication
 - A single instruction multiply-accumulates matrices
 - Computes many dot-products in parallel

Source: EE290-2 Shao

/ Bfloat16 for Google's Tensor Processing Unit

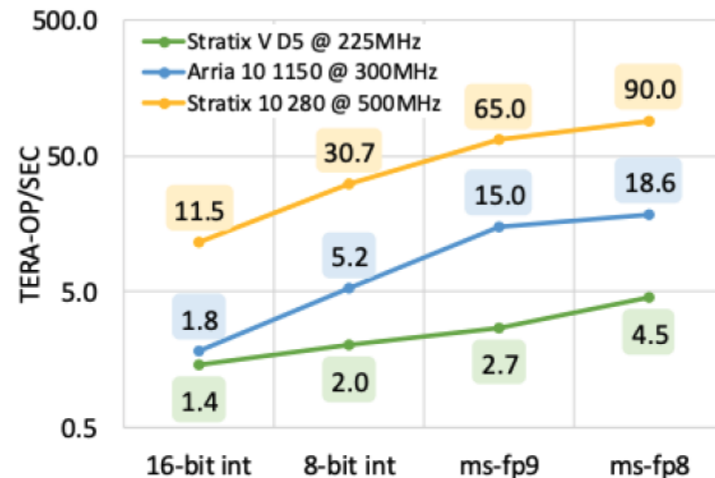


- fp32 - IEEE single-precision floating-point
- fp16 - IEEE half-precision floating point
- bfloat16 - 16-bit *brain floating point*



MSFP in Brainwave FPGA @ Microsoft

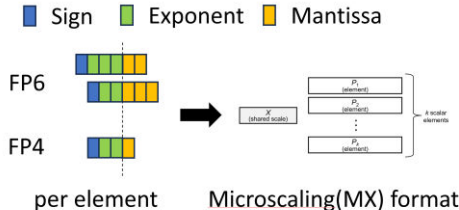
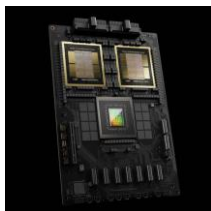
- “ ‘neural’-optimized data formats based on 8- and 9-bit floating point, where mantissas are trimmed to 2 or 3 bits. “
- “ These formats, referred to as ms-fp8 and ms-fp9, exploit efficient packing into reconfigurable resources and are comparable in FPGA area”



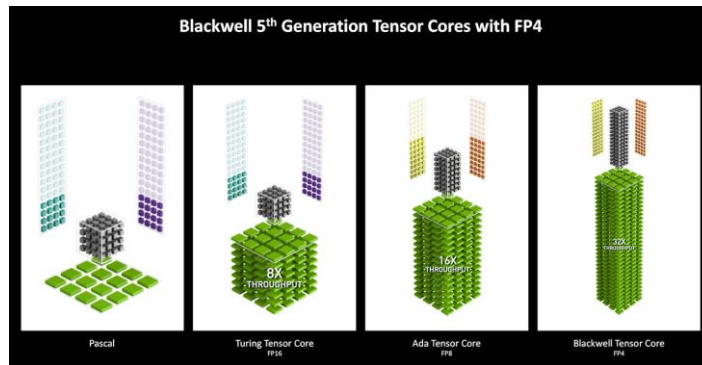
GPU tensor cores

E.g. Newest Generation

Nvidia DGX B200 GPU



new data format



lower bit-width, higher throughput

- <https://www.nvidia.com/en-us/data-center/dgx-b200/>
- New features:
 - FP4/6 format tensor core support (FP16->BF16->FP8->FP4/6)
 - Chiplet design – two dies with chip-to-chip interconnect in a unified single GPU
 - HW/SW co-design: 2nd Transformer Engine & Microscaling format

Promising Low-bit-width Quantization



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Image Generation on Blackwell Silicon

Using FP4-quantized model



Model using FP16



Model using FP4

Prompt: Close up photo of a rabbit, forest in spring, haze, halation, bloom, dramatic atmosphere, centered, rule of thirds, 200mm 1.4f macro shot

Outputs generated using SDXL 1.0 Base. FP4 result generated with NVIDIA Quasar Quantization System.

/复习小结



- Fixed-point and floating-point representations
- Hardware implications
- Quantization methods for deep learning
- Hardware support for low precision DNNs

布置作业



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

1) Reading for this week

- Quantization Paper
- Submit your review before next course.

2) Lab 1 will be posted after next course

- A quantized network on cifar10

3) 看量化报告讲座

- 网络学堂上YouTub视频链接



加州大学伯克利分校 EE290-2课程 Sophia Shao教授

寒武纪 智能计算系统 课程

三位助教帮助完成课件整理

/Reference



- Zhou S, Wu Y, Ni Z, et al. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients[J]. arXiv preprint arXiv:1606.06160, 2016.
- Zhou A, Yao A, Guo Y, et al. Incremental network quantization: Towards lossless cnns with low-precision weights[J]. arXiv preprint arXiv:1702.03044, 2017.
- Choi J, Wang Z, Venkataramani S, et al. Pact: Parameterized clipping activation for quantized neural networks[J]. arXiv preprint arXiv:1805.06085, 2018.
- Park E, Kim D, Yoo S. Energy-efficient neural network accelerator based on outlier-aware low-precision computation[C]//2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA). IEEE, 2018: 688-698.
- Jung S, Son C, Lee S, et al. Learning to quantize deep networks by optimizing quantization intervals with task loss[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 4350-4359.
- Berkeley open course resources EE290